

AI Engineer — Assignment

AI-Powered Quiz Generation using RAG

Learnyst Engineering

Role: AI Engineer

Objective

Build a production-ready backend service that generates quizzes from lesson transcripts using a Retrieval-Augmented Generation (RAG) pipeline.

This assignment evaluates your ability to design and build AI applications that leverage vector databases, embeddings, retrieval, and Large Language Models (LLMs) to generate high-quality, context-aware quizzes.

Problem Statement

You will be provided with one or more lesson transcripts in plain text format. Your task is to build a backend service that:

- Ingests lesson transcripts.
- Chunks and embeds the content.
- Stores embeddings in a vector database.
- Retrieves relevant context.
- Generates quizzes using an LLM.

The generated questions **must be based only on the provided lesson transcript**.

Technical Requirements

You may use any technology stack. Suggested options:

- **Language:** Node.js (TypeScript) or Python
- **LLM:** Gemini, OpenAI, Claude, Ollama, etc.
- **Vector DB:** pgvector, ChromaDB, FAISS, Pinecone, Weaviate, Qdrant, or Milvus
- **Embeddings:** any embedding model

The implementation choices are yours and should be justified during the interview.

Getting Started — Free Models & API Keys

The entire assignment can be completed on free-tier services — no paid account or credit card required. The stack below is the recommended path:

Layer	Recommended (free)
Quiz generation (LLM)	Gemini 2.5 Flash · model id: gemini-2.5-flash
Embeddings	Gemini Embedding · model id: gemini-embedding-001
Vector database	Chroma, FAISS, Qdrant, or pgvector (local / free)

Both `gemini-2.5-flash` and `gemini-embedding-001` are available on the Gemini API free tier, so no billing setup is needed. A newer multimodal `gemini-embedding-2` is also available if you prefer.

Generate a Free Gemini API Key

1. Open Google AI Studio: aistudio.google.com
2. Sign in with your Google account.
3. Click **Get API key** (or open aistudio.google.com/apikey).
4. Click **Create API key**.
5. Copy the generated API key.
6. Create a `.env` file in your project root and add the key:

```
GEMINI_API_KEY=your_api_key_here
```

API Requirements

1. Ingest Lesson

Endpoint: POST `/ingest`

Request

```
{
  "lessonId": "lesson-001",
  "content": "<lesson transcript>"
}
```

Expected behaviour — the API should accept the transcript, chunk the content, generate embeddings, store the chunks and embeddings in a vector database, and store relevant metadata (e.g. `lessonId`, `chunkIndex`, `chunkText`). You are free to design your own schema.

2. Generate Quiz

Endpoint: POST `/generate-quiz`

Request

```
{
  "lessonId": "lesson-001",
  "questionCount": 10,
  "difficulty": "medium",
  "questionTypes": [
    "mcq",
    "true_false",
    "fill_blank",
    "short_answer"
  ]
}
```

Expected behaviour — the API should retrieve relevant chunks from the vector database, construct the prompt using the retrieved context, generate quiz questions, and return structured JSON. Questions must be generated only from the lesson content.

Output Format

Each question should contain: question, question type, difficulty, correct answer, explanation, and options (for MCQs).

Example

```
{
  "question": "What is React?",
  "type": "mcq",
  "difficulty": "easy",
  "options": ["Library", "Framework", "Database", "Browser"],
  "correctAnswer": "Library",
  "explanation": "React is a JavaScript library used for building user interfaces."
}
```

Frontend & Deployment

User Interface (UI)

Build a basic frontend for the quiz generator. Requirements are intentionally minimal:

- At minimum, a single **Generate** button that calls the generate-quiz API and displays the returned questions.
- Render each question clearly — question, type, options, correct answer, and explanation.
- Use any framework, library, or styling you prefer; the design is entirely your choice.

Hosting

Deploy the project to any free hosting service so it is reachable at a public URL (e.g., Vercel, Netlify, Render, Railway, or Hugging Face Spaces). Share the live URL together with your GitHub repository.

Functional Requirements

- Generate questions only from the provided lesson transcript.
- Support multiple question types.
- Support Easy, Medium, and Hard difficulty.

- Return valid JSON.
- Avoid duplicate questions.
- Avoid duplicate answer choices.
- Include explanations for every question.
- Handle invalid LLM responses gracefully.

Error Handling

Handle scenarios such as:

- Invalid request payload
- Missing lesson / empty transcript
- Invalid JSON returned by the LLM
- Embedding failures
- Vector database failures
- LLM failures and timeout scenarios

Deliverables

- Live hosted URL (deployed on a free hosting service)
- GitHub repository
- README with setup instructions
- Sample requests and sample responses

The project should run locally with minimal setup.

Bonus (Optional)

Implement any of the following:

- Streaming responses
- Bloom's Taxonomy support
- Export quizzes as JSON or CSV

Evaluation Criteria

Criteria	Weight
Code Quality & Architecture	20%
RAG Pipeline Design	20%
Prompt Engineering & LLM Integration	20%
Quiz Quality & Accuracy	15%
Vector Database & Retrieval Strategy	10%
Error Handling & Validation	10%
Documentation	5%

Submission

Please submit: the live hosted URL, a GitHub repository link, instructions to run the project locally, and any assumptions or trade-offs made during implementation.

Notes: You are free to choose any libraries, frameworks, vector database, embedding model, or LLM. We are more interested in your design decisions, code quality, and reasoning than the specific technologies you choose. Keep the implementation simple, clean, and production-oriented.